

점프 투 자바

박응용 지음(위키독스 운영자)

이지스퍼블리싱

06

자바의 입출력



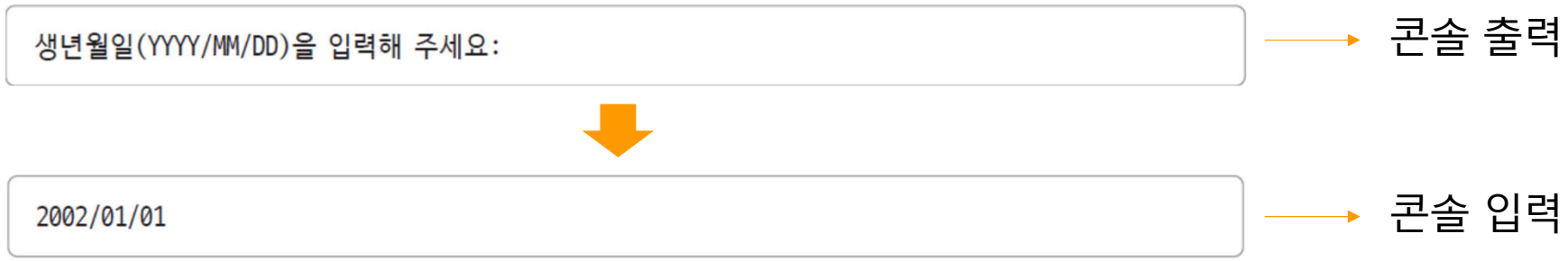
06-1 콘솔 입출력

06-2 파일 입출력

06-1 콘솔 입출력

▪ 콘솔 입력

- 사용자가 다음과 같이 출력된 질문에 키보드로 답변을 입력하는 것



06-1 콘솔 입출력

▪ 콘솔 입력

- 사용자가 입력한 문자열을 얻으려면 System.in 사용

Do it! 코딩해 보세요

```
import java.io.IOException;
import java.io.InputStream;

public class Sample {
    public static void main(String[] args) throws IOException {
        InputStream in = System.in;

        int a;
        a = in.read();

        System.out.println(a);
    }
}
```

- InputStream은 자바의 내장 클래스
  - 필요할 때마다 임포트(import)하여 사용
- 이 프로그램은 사용자의 입력을 대기하다 사용자가 입력한 문자에 해당하는 아스키코드값을 출력함

실행 결과

```
97
```

06-1 콘솔 입출력

▪ 콘솔 입력

- **InputStream**
  - 바이트(byte) 단위의 데이터를 읽어 들일 때 사용하는 내장 클래스
  - InputStream의 read 메서드는 1byte만 읽어 아스키코드값으로 출력
  - 입력 스트림 – 사용자가 전달한 데이터. byte의 흐름을 추상화한 개념이며 데이터를 byte 단위로 다룸



06-1 콘솔 입출력

▪ 콘솔 입력

▪ InputStream

- 사용자가 3byte를 입력했을 때 3byte를 전부 읽는 코드

Do it! 코딩해 보세요

```
import java.io.IOException;
import java.io.InputStream;

public class Sample {
   public static void main(String[] args) throws IOException {
      InputStream in = System.in;

      int a;
      int b;
      int c;

      a = in.read();
      b = in.read();
      c = in.read();

      System.out.println(a);
      System.out.println(b);
      System.out.println(c);
   }
}
```

실행 결과

```
abc   ← abc를 입력한 후, Enter 키를 누른다.
97
98    ← 출력
99
```

06-1 콘솔 입출력

▪ 콘솔 입력

- **InputStream**
  - 사용자가 3byte를 입력했을 때 3byte를 전부 읽는 코드
    - 길이가 3byte인 배열을 만든 후 read 메서드의 입력값으로 전달

Do it! 코딩해 보세요

```
import java.io.IOException;
import java.io.InputStream;

public class Sample {
   public static void main(String[] args) throws IOException {
      InputStream in = System.in;

      byte[] a = new byte[3];
      in.read(a);

      System.out.println(a[0]);
      System.out.println(a[1]);
      System.out.println(a[2]);
   }
}
```

실행 결과

```
abc   ← abc를 입력한 후, Enter 키를 누른다.
97
98    ← 출력
99
```

06-1 콘솔 입출력

▪ 콘솔 입력

▪ InputStreamReader

- 사용자가 입력한 문잣값을 그대로 출력할 때 사용

Do it! 코딩해 보세요

```
import java.io.IOException;
import java.io.InputStream;
import java.io.InputStreamReader;

public class Sample {
    public static void main(String[] args) throws IOException {
        InputStream in = System.in;
        InputStreamReader reader = new InputStreamReader(in);
        char[] a = new char[3];
        reader.read(a);

        System.out.println(a);
    }
}
```

실행 결과

```
abc   <- abc를 입력한 후, Enter 키를 누른다.
abc   <- 출력
```

06-1 콘솔 입출력

▪ 콘솔 입력

- **BufferedReader**
  - 길이에 상관없이 사용자가 입력한 문잣값을 모두 읽어 들일 때 사용

기억 방법

- InputStream: byte를 읽는다.
- InputStreamReader: character(문자)를 읽는다.
- BufferedReader: String(문자열)을 읽는다.

Do it! 코딩해 보세요

```
import java.io.IOException;
import java.io.BufferedReader;
import java.io.InputStream;
import java.io.InputStreamReader;

public class Sample {
    public static void main(String[] args) throws IOException {
        InputStream in = System.in;
        InputStreamReader reader = new InputStreamReader(in);
        BufferedReader br = new BufferedReader(reader);

        String a = br.readLine();
        System.out.println(a);
    }
}
```

실행 결과

```
Hello World   ← abc를 입력한 후, Enter 키를 누른다.
Hello World   ← 출력
```

06-1 콘솔 입출력

▪ 콘솔 입력

- **Scanner**
  - 다양한 값을 읽어 들이는 메서드와 함께 사용
    - next: 토큰을 읽어 들일 수 있다.
    - nextLine: 라인을 읽어 들일 수 있다.
    - nextInt: 정수를 읽어 들일 수 있다.

Do it! 코딩해 보세요

```
import java.util.Scanner;

public class Sample {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println(sc.next());
        sc.close();
    }
}
```

06-1 콘솔 입출력

콘솔 출력

- System.out.println 메서드
  - 콘솔에 문자열을 출력할 때나 디버깅할 때 많이 사용
- System.err 메서드
  - 오류 메시지를 출력할 때 사용
  - System.out과 동일한 역할

06-2 파일 입출력

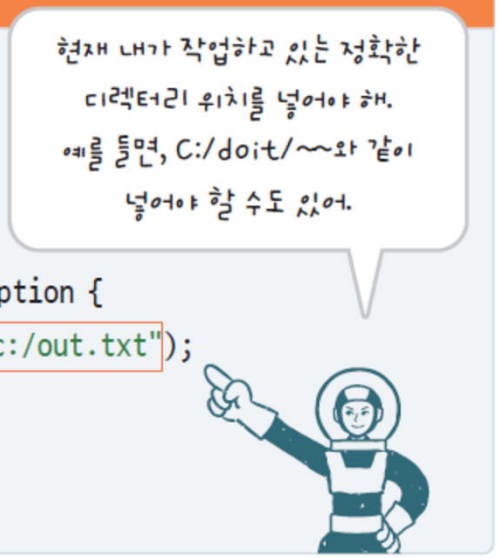
▪ 파일 쓰기

- c:/ 디렉터리 바로 밑에 새로운 파일인 out.txt를 생성하는 예제
  - FileOutputStream 클래스 사용

Do it! 코딩해 보세요

```
import java.io.FileOutputStream;
import java.io.IOException;

public class Sample {
    public static void main(String[] args) throws IOException {
        FileOutputStream output = new FileOutputStream("c:/out.txt");
        output.close();
    }
}
```



06-2 파일 입출력

▪ 파일 쓰기

- **FileOutputStream**
  - OutputStream 클래스를 상속받아 만든 클래스이므로 byte 단위로 데이터를 처리
  - String을 byte 배열로 바꾸어 주는 getBytes() 메서드 이용

Do it! 코딩해 보세요

```
import java.io.FileOutputStream;
import java.io.IOException;

public class Sample {
    public static void main(String[] args) throws IOException {
        FileOutputStream output = new FileOutputStream("C:/out.txt");
        for(int i=1; i<11; i++) {
            String data = i+" 번째 줄입니다.\r\n";
            output.write(data.getBytes());
        }
        output.close();
    }
}
```

06-2 파일 입출력

파일 쓰기

- **FileWriter**
  - byte 배열 대신 문자열을 사용할 수 있어 편리함

Do it! 코딩해 보세요

```
import java.io.FileWriter;
import java.io.IOException;

public class Sample {
    public static void main(String[] args) throws IOException {
        FileWriter fw = new FileWriter("C:/out.txt");
        for(int i=1; i<11; i++) {
            String data = i+" 번째 줄입니다.\r\n";
            fw.write(data);
        }
        fw.close();
    }
}
```

06-2 파일 입출력

파일 쓰기

- **PrintWriter**
  - 문자열 뒤에 ₩r₩n을 덧붙이는 대신 println 메서드 사용

Do it! 코딩해 보세요

```
import java.io.IOException;
import java.io.PrintWriter;

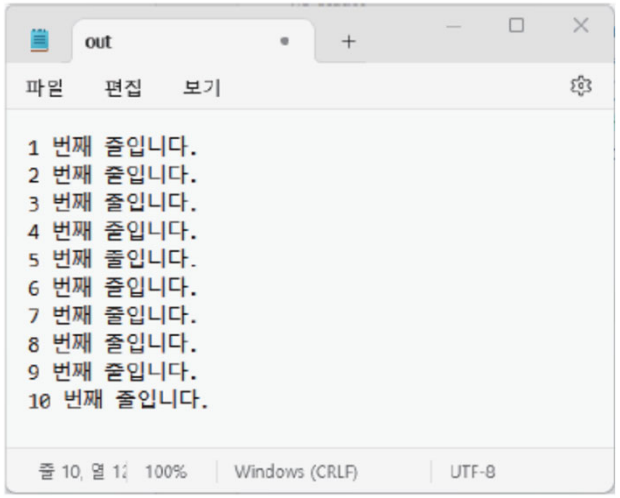
public class Sample {
    public static void main(String[] args) throws IOException {
        PrintWriter pw = new PrintWriter("c:/out.txt");
        for(int i=1; i<11; i++) {
            String data = i+" 번째 줄입니다.";
            pw.println(data);
        }
        pw.close();
    }
}
```

06-2 파일 입출력

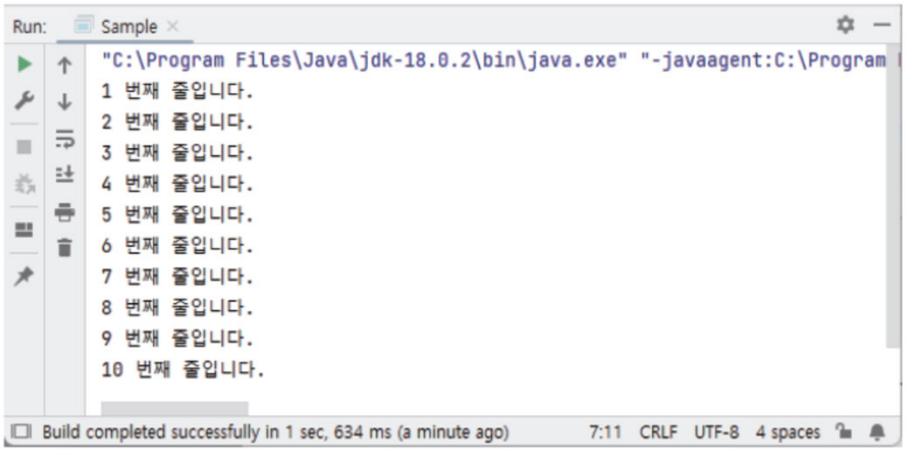
▪ 파일 쓰기

▪ PrintWriter

- PrintWriter를 사용한 out.txt 파일 출력 화면



- System.out을 사용한 콘솔 출력 화면



06-2 파일 입출력

▪ 파일 쓰기

▪ 파일에 내용 추가하기

- 이미 작성된 파일을 추가 모드로 열어서 추가할 내용을 작성

Do it! 코딩해 보세요

```
import java.io.FileWriter;
import java.io.IOException;

public class Sample {
    public static void main(String[] args) throws IOException {
        FileWriter fw = new FileWriter("c:/out.txt");
        for(int i=1; i<11; i++) {
            String data = i+" 번째 줄입니다.\r\n";
            fw.write(data);
        }
        fw.close();
```

- 기존 파일에 내용을 덮어쓰지 않고 이어 쓰게 됨
- FileWriter를 이용하는 예

```
        FileWriter fw2 = new FileWriter("c:/out.txt", true);  // 파일을 추가 모드로 연다.
        for(int i=11; i<21; i++) {
            String data = i+" 번째 줄입니다.\r\n";
            fw2.write(data);
        }
        fw2.close();
    }
}
```

06-2 파일 입출력

▪ 파일 쓰기

▪ 파일에 내용 추가하기

- PrintWriter를 이용하는 예

Do it! 코딩해 보세요

```
import java.io.FileWriter;
import java.io.IOException;
import java.io.PrintWriter;

public class Sample {
    public static void main(String[] args) throws IOException {
        PrintWriter pw = new PrintWriter("c:/out.txt");
        for(int i=1; i<11; i++) {
            String data = i+" 번째 줄입니다.";
            pw.println(data);
        }
        pw.close();
```

- 생성자의 파라미터로 파일명 대신
  추가 모드로 열린 FileWriter의 객체를 전달해야 함

```
        PrintWriter pw2 = new PrintWriter(new FileWriter("c:/out.txt", true));
        for(int i=11; i<21; i++) {
            String data = i+" 번째 줄입니다.";
            pw2.println(data);
        }
        pw2.close();
    }
}
```

06-2 파일 입출력

▪ 파일 읽기

- FileInputStream 클래스 이용
- byte 배열을 이용하여 파일을 읽어야 하기 때문에 읽어야 하는 데이터 길이를 모를 경우에는 불편한 방법

Do it! 코딩해 보세요

```
import java.io.FileInputStream;
import java.io.IOException;
```

```
public class Sample {
    public static void main(String[] args) throws IOException {
        byte[] b = new byte[1024];
        FileInputStream input = new FileInputStream("c:/out.txt");
        input.read(b);
        System.out.println(new String(b));  // byte 배열을 문자열로 변경하여 출력
        input.close();
    }
}
```

06-2 파일 입출력

▪ 파일 읽기

- 한 줄 단위로 파일을 읽으려면 FileReader와 BufferedReader의 조합 사용
- BufferedReader의 readLine 메서드는 더 이상 읽을 라인이 없을 경우 null을 리턴

Do it! 코딩해 보세요

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

public class Sample {
    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new FileReader("c:/out.txt"));
        while(true) {
            String line = br.readLine();
            if (line==null) break;  // 더 이상 읽을 라인이 없을 경우 while 문을 빠져나간다.
            System.out.println(line);
        }
        br.close();
    }
}
```



감사합니다.